GUÍA PRÁCTICA FORMATIVA: ESTRATEGIA DE DESPLIEGUE CANARY RELEASES

Ingeniería de Software V — Sesión 22 (Semana 11)

Enrutamiento Porcentual Ponderado con Nginx, Telemetría, Mitigación de Blast Radius y Rollback

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	**PONDERACIÓN:**	**Formativa** (Sin calificación sumativa) • Duración: 2 Horas
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**MODALIDAD:**	Parejas / Laboratorio Práctico Guiado
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto del Problema & Arquitectura Canary

1.1 Descripción del Reto de Despliegue

Mientras que la estrategia Blue-Green conmuta el 100 % de los usuarios de un solo golpe, ciertas fallas críticas (ej. fugas de memoria progresivas, degradación de latencia bajo carga real o incompatibilidades con casos borde de clientes) solo se manifiestan tras varios minutos de tráfico continuo. Si se conmuta al 100 %, la totalidad de los usuarios sufren el impacto del incidente.

La estrategia **Canary Release** minimiza drásticamente el radio de impacto (*Blast Radius*) mediante una introducción gradual del nuevo código:

- **Enrutamiento Porcentual Inicial:** La versión estable (`v1.0.0`) retiene el 90 % del tráfico, mientras que la versión candidata (`v2.0.0 / Canary`) solo recibe el 10 %.
- **Criterios Deterministas de Ruteo:** Nginx aplica el módulo `split_clients` mediante una función hash sobre la dirección IP o la cabecera HTTP del cliente, garantizando que una sesión continúe en la misma versión.
- **Monitoreo de Telemetría y SLOs:** Si la versión Canary reporta una tasa de errores HTTP 5xx o latencias elevadas, se corta el tráfico inmediatamente sin afectar al 90 % restante.
- **Promoción Progresiva:** Si los indicadores de telemetría son óptimos, el tráfico se incrementa gradualmente ($10\,\% \rightarrow 50\,\% \rightarrow 100\,\%$) hasta retirar la versión anterior.

Componente	Tecnología & Contenedor	Puerto / Endpoint
Canary Gateway	Nginx 1.25 Alpine (`split_clients`)	Puerto `8080` (`http://localhost:8080`)
Versión Stable	Microservicio v1.0.0 (`app-stable`)	Puerto interno `8081` (90 % Tráfico)
Versión Canary	Microservicio v2.0.0 (`app-canary`)	Puerto interno `8082` (10 % Tráfico inicial)
Inyección de Fallos	Endpoint simulador (`/api/fault`)	Disparador de errores HTTP 500

2 Procedimiento Experimental Paso a Paso

2.1 Fase 1: Puesta en Marcha del Stack Canary (15 min)

Ubíquese en `codigo_base/` e inicie los tres contenedores configurados en `docker-compose.yml`:

```
docker compose up -d --build
docker compose ps
```

2.2 Fase 2: Configuración del Enrutamiento Porcentual en Nginx (20 min)

Inspeccione el archivo `nginx/nginx.conf`. Note cómo la directiva `split_clients` distribuye el tráfico:

```
events { worker_connections 1024; }

http {
    # Mapeo determinista por IP de cliente
    split_clients "${remote_addr}AAA" $backend_upstream {
        10%     app-canary:8080;    # 10% del trafico
        *       app-stable:8080;    # 90% restante
    }

    server {
        listen 80;
        location / {
            proxy_pass http://$backend_upstream;
            proxy_set_header Host $host;
            add_header X-Served-By $backend_upstream always;
        }
    }
}
```

2.3 Fase 3: Verificación Estadística de la Distribución de Tráfico (25 min)

Ejecute el script de prueba sintética `./test_traffic.sh`, el cual envía 100 peticiones simulando diferentes clientes y tabula los resultados:

```
./test_traffic.sh 100
```

Observe la salida estadística consolidada:

- **Stable (v1.0.0):** $\sim 90\,\%$ de las respuestas con cabecera `X-Served-By: app-stable:8080`.
- **Canary (v2.0.0):** $\sim 10\,\%$ de las respuestas con cabecera `X-Served-By: app-canary:8080`.

2.4 Fase 4: Simulación de Degradación y Alerta de Telemetría (25 min)

Simule la introducción de un bug crítico en la versión Canary activando la inyección de fallos:

```
# Provocar que el contenedor Canary responda HTTP 500 ante peticiones:
curl -X POST http://localhost:8082/api/fault/enable
```

Ejecute nuevamente el monitor de tráfico:

```
./test_traffic.sh 50
```

Verifique que los errores HTTP 500 solo afectan al $\sim 10\,\%$ de las peticiones, mientras que el $90\,\%$ de los clientes continúan operando con normalidad sobre Stable.

2.5 Fase 5: Rollback y Evacuación Inmediata de Tráfico (15 min)

Ante la alarma de fallos en producción, evacúe el tráfico de Canary inmediatamente ejecutando:

```
./rollback.sh
```

El script ajusta la ponderación en Nginx a 0 % para Canary y recarga en caliente (`nginx -s reload`). Vuelva a enviar peticiones y compruebe que el 100 % del tráfico regresa a Stable con 0 errores.

2.6 Fase 6: Promoción Exitosa a Producción (Opcional - 20 min)

Restaure el estado saludable del Canary (`curl -X POST http://localhost:8082/api/fault/disable`) y ejecute el script de promoción progresiva:

```
./promote_canary.sh 50   # Incrementar a 50%
./promote_canary.sh 100  # Promocionar al 100% (Canary se convierte en la nueva Stable)
```

3 Cuestionario de Reflexión Conceptual & Checklist Formativo

3.1 Preguntas de Reflexión Técnica

1. **Blast Radius (Radio de Impacto):** Compare el impacto que tiene un error grave (ej. fallo en procesamiento de tarjetas) desplegado con Blue-Green (100 % de usuarios) frente a un Canary del 5 %. ¿Cómo protege esto la métrica MTTR (Mean Time to Recover)?
2. **Criterios de Telemetría Automatizada:** ¿Por qué las organizaciones avanzadas (ej. Spinnaker / Argo Rollouts) automatizan el rollback mediante análisis estadístico de Prometheus en lugar de esperar la revisión manual de un ingeniero?
3. **Afinidad de Sesión en Canary:** Si un usuario agrega un artículo al carrito en la versión Canary y en la siguiente petición cae en Stable, ¿qué problemas de experiencia surgen si no existe persistencia compartida?

3.2 Checklist de Logro Formativo (Autoevaluación)

Objetivo Formativo	Evidencia Observable	Estado
1. Enrutamiento Ponderado	Nginx distribuye 90 % a Stable y 10 % a Canary de forma determinista.	[ ] Logrado
2. Medición Estadística	Script de tráfico confirma distribución porcentual de respuestas.	[ ] Logrado
3. Mitigación de Blast Radius	Fallo inyectado en Canary solo afecta la cuota asignada (10 %).	[ ] Logrado
4. Evacuación / Rollback	Tráfico conmutado a 100 % Stable con `nginx -s reload` sin caídas.	[ ] Logrado
5. Promoción Progresiva	Incremento de tráfico a 50 % y 100 % verificado con telemetría.	[ ] Logrado